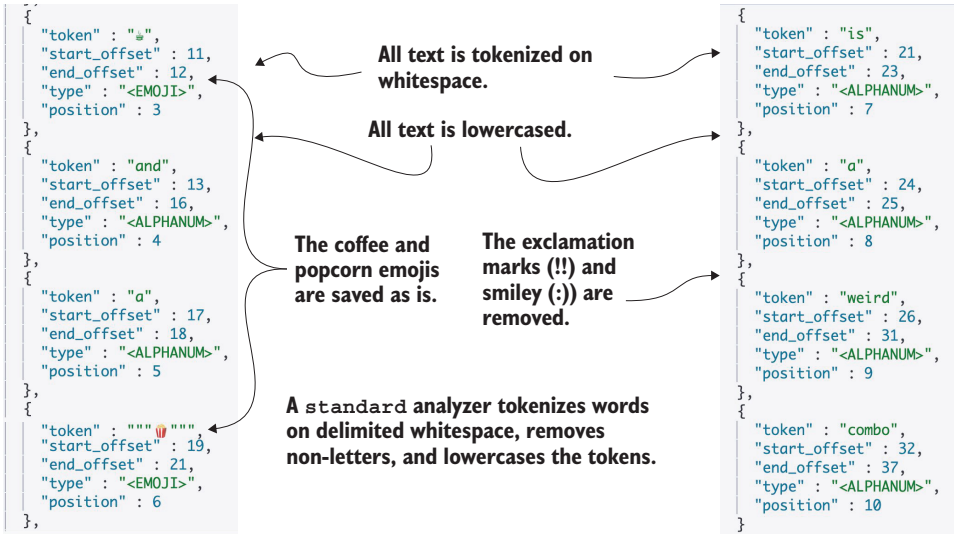


Figure 7.5 The output tokens from a `standard` analyzer

As you can see, the tokens for "coffee" and "popcorn" are stored as is, and non-letter characters such as ":(" and "!!" are removed. The "coffee" token has a width of one character (check the offsets), and the "popcorn" token is stored as two characters wide.

TESTING THE STANDARD ANALYZER

We can add a specific analyzer during our text analysis testing phase by adding an `analyzer` attribute in the code. The following listing demonstrates this.

Listing 7.4 Testing the `standard` analyzer with an explicit call

```
GET _analyze
{
  "analyzer": "standard",
  "text": "Hot cup of ☕ and a 🍿 is a Weird Combo :(!!"
}
```

Specifies the analyzer (not really necessary here because standard is the default analyzer)

This code produces the result shown in figure 7.5. We can replace the value of the analyzer with a chosen one if we are testing the `text` field using a different analyzer: for example, `"analyzer": "whitespace"`.

The output indicates that the text was tokenized and lowercased.

Figure 7.6 illustrates the `standard` analyzer with its internal components and anatomy. It consists of a `standard` tokenizer and two token filters: `lowercase` and `stop`. No character filter is defined on the `standard` analyzer. Remember, analyzers consist of zero or more character filters, at least one tokenizer, and zero or more token filters.